UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA

Marija Antić
Miloš Pilipović

PROJEKTOVANJE IoT SISTEMA

Praktikum za laboratorijske vežbe



Novi Sad, 2020.

PREDGOVOR

Knjiga koja je pred vama predstavlja dopunu udžbeniku *Bežične mreže i Internet stvari*, i namenjena je upotrebi na laboratorijskim vežbama. Dok osnovni udžbenik izlaže fundamentalne koncepte bežičnih komunikacija, te daje pregled aktuelnih tehnologija koje se koriste u IoT oblasti, praktikum ima za cilj da identifikuje osnovne koncepte na kojima se zasniva projektovanje IoT sistema i omogući realizaciju jednostavnog primera takvog sistema na laboratorijskim vežbama, tokom jednog semestra.

Izrada vežbi planirana je u grupama (projektnim timovima), tako da je obim zadataka tome i prilagođen. Planirane su četiri kontrolne tačke tokom rada, posle trećeg, šestog, osmog i jedanaestog termina vežbi, uz mogućnost nadoknade tokom jednog od termina. Kontrolne tačke odgovaraju fazama u razvoju funkcionalnosti sistema i na kraju svake od tih faza treba demonstrirati stanje sistema.

Tokom početne tri vežbe cilj je razumeti komponente koje čine jedan IoT sistem i osmisliti arhitekturu sistema zadate namene. Na kraju ove faze rada potrebno je prezentovati specifikaciju sistema, po kojoj će projektni tim dalje raditi.

Drugu fazu razvoja predstavlja rešavanje problema vezanih za uspostavljanje i održavanje IoT mreže. U trećoj fazi razvoja bavićemo se funkcionalnošću samih IoT uređaja, dok je četvrta faza razvoja sistema rezervisana za razvoj naprednih funkcionalnosti kontrolera mreže.

SADRŽAJ

PREDGOVOR ..... **I**

SADRŽAJ ..... **II**

1. ARHITEKTURA IOT SISTEMA **5**

1.1. KOMPONENTE IOT SISTEMA 5
1.1.1. Krajnji uređaji *5*
1.1.2. Kontroler sistema *5*
1.1.3. Korisničke aplikacije *6*
1.1.4. Servisi u oblaku *6*
1.2. REFERENTNI PRIMER: SISTEM ZA KONTROLU PAMETNE KUĆE 6
ZADATAK VEŽBE 1 9

2. MODEL UREĐAJA **10**

2.1. MODELI UREĐAJA U POPULARNIM IOT TEHNOLOGIJAMA 10
2.1.1. Zigbee *10*
2.1.2. Z-Wave *11*
2.1.3. BLE *12*
2.2. OGLAŠAVANJE FUNKCIONALNOSTI UREĐAJA 13
2.3. IZVRŠAVANJE KOMANDI 14
ZADATAK VEŽBE 2 15

3. KOMUNIKACIONI MODEL **16**

3.1. MQTT PROTOKOL 16
3.2. OBRASCI ZA KOMUNIKACIJU 16
3.2.1. Asinhrona komunikacija (mehanizam pretplate i objave) *16*
3.2.2. Sinhrona komunikacija (mehanizam zahteva i odgovora) *17*
3.2.3. Teme u MQTT protokolu *17*
3.2.4. Struktura poruka *17*
3.2.5. Kvalitet nivoa usluge(QoS) *18*
ZADATAK VEŽBE 3 19

KONTROLNA TAČKA 1 ..... **20**

4. OTKRIVANJE MREŽE I PRIKLJUČIVANJE UREĐAJA **21**

4.1. NAČINI OGLAŠAVANJA U POPULARNIM IOT TEHNOLOGIJAMA 21

4.2. OGLAŠAVANJE WIFI MREŽE 21
4.3. SSDP PROTOKOL 21
4.3.1. NOTIFY poruka 22
4.3.2. M-SEARCH poruka 24
4.3.3. RESPONSE poruka kao odgovor na M-SEARCH poruku 24
4.4. OPIS FUNKCIONALNOSTI UREĐAJA 25
ZADATAK VEŽBE 4A 27
ZADATAK VEŽBE 4B 28

5. ODRŽAVANJE MREŽE 29

5.1. PRAĆENJE STANJA MREŽE I UREĐAJA NA STRANI KONTROLERA 29
5.1.1. Čuvanje informacija o uređajima u mreži 29
5.1.2. Otkrivanje nedostupnog uređaja 29
5.2. NAČINI RADA UREĐAJA 29
5.2.1. Spavajući uređaji 29
5.2.2. Uvek aktivni uređaji 30
ZADATAK VEŽBE 5A 31
ZADATAK VEŽBE 5B 32
ZADATAK VEŽBE 5C 33

KONTROLNA TAČKA 2 34

6. UPRAVLJANJE UREĐAJIMA 35

6.1. FUNKCIONALNOST UREĐAJA 35
6.1.1. Izvršavanje komandi 35
6.1.2. Odgovor na upit stanja 35
6.2. KORISNIČKA APLIKACIJA 36
6.2.1. Prikaz trenutnog stanja uređaja 36
6.2.2. Izvršavanje komande 37
6.3. ULOGA KONTROLERA 37
6.3.1. Keširanje informacije o trenutnom stanju uređaja 37
ZADATAK VEŽBE 6A 38
ZADATAK VEŽBE 6B 39
ZADATAK VEŽBE 6C 40

KONTROLNA TAČKA 3 41

7. NAPREDNA LOGIKA KONTROLERA 42

7.1. GRUPISANJE UREĐAJA 42
7.2. AUTOMATSKA PODEŠAVANJA 42
7.2.1. Periodično izvršavanje komandi 42
7.2.2. Uslovno izvršavanje komandi 42
ZADATAK 7A 43

ZADATAK VEŽBE 7B..... 44
ZADATAK VEŽBE 7C..... 45
KONTROLNA TAČKA 4..... 46
LITERATURA..... 47

1. ARHITEKTURA IOT SISTEMA

1.1. Komponente IoT sistema

Tipično, IoT sistem integriše veliki broj komponenti:

1.1.1. Krajnji uređaji

i) Senzori

ii) Aktuatori

- Senzori – prate i prijavljuju promenu određenih fizičkih veličina
  - temperatura
  - vlažnost vazduha
  - vlažnost zemljišta
  - detekcija pokreta
  - osvetljenost
- Aktuatori – reaguju na komande koje dolaze sa aplikativnog sloja i izvršavaju neku akciju
  - paljenje/gašenje svetla
  - postavljanje željene vrednosti temperature na termostatu
  - pokretanje/zaustavljanje kretanja roletni
  - uključivanje/isključivanje pumpe za vodu

1.1.2. Kontroler sistema

i) Lokalni kontroler

ii) Kontroler u oblaku

- Kontroler sistema
  - Uređaj koji upravlja čitavim sistemom
  - omogućava priključivanje krajnjih uređaja mreži
  - omogućava uzročno-posledične veze među promenama u sistemu (npr. uključivanje klimatizacije onda kada temperatura očitana sa senzora pređe zadatu vrednost)
  - realizuje logiku upravljanja aktuatorima na bazi senzora i informacija sa interneta

- Nije obavezno postojanje kontrolera kao posebnog uređaja, deo ovih funkcionalnosti može biti u okviru servisa u oblaku (IFTTT servis, na primer)

1.1.3. Korisničke aplikacije

- Omogućavaju kontrolu sistema od strane korisnika (Android, iOS, web klijent)

1.1.4. Servisi u oblaku

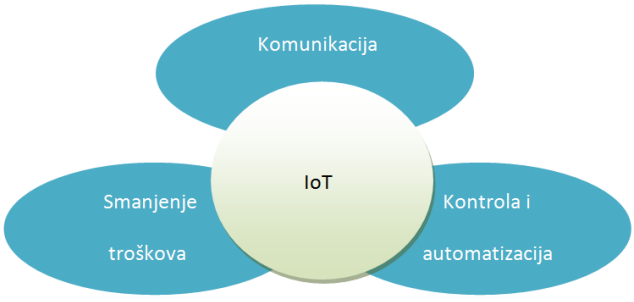
- Servisi u okviru oblaka datog IoT sistema
  - registracija korisnika
  - ažuriranje verzija softvera na kontrolerima ili krajnjim uređajima
  - komunikacija korisničke aplikacije sa kontrolerima ili krajnjim uređajima koji se nalaze van njene lokalne mreže
- Integracija sa drugim servisima koji donose vrednost za korisnika
  - vremenska prognoza
  - stanje saobraćaja
  - geolokacija
  - prepoznavanje govora

Komunikacija među komponentama sistema odvija se različitim protokolima: *Zigbee, Zwave, Bluetooth, IP, MQTT, AMQP,* itd.

1.2. Referentni primer: sistem za kontrolu pametne kuće

Šta se podrazumeva pod pojmom pametne kuće?

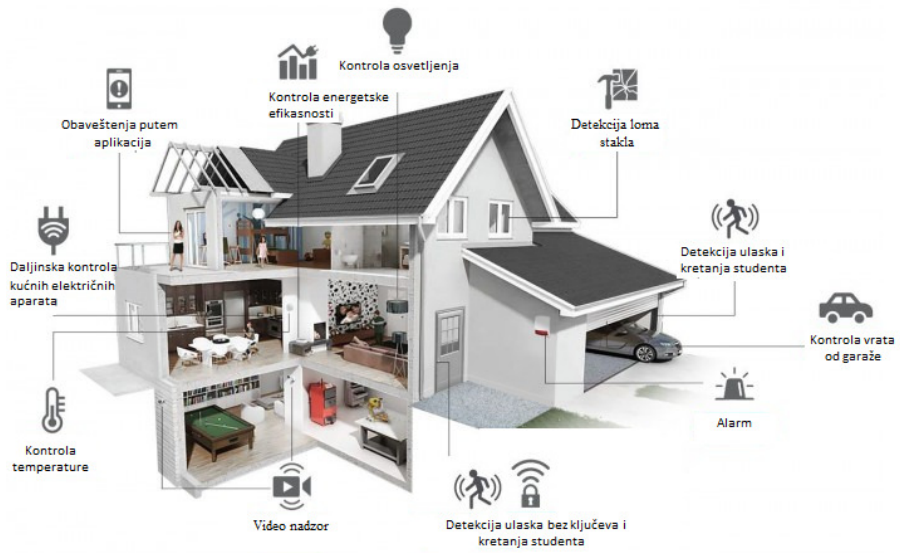
To je kuća koja integriše veliki broj prethodno navedenih komponenti IoT sistema u poseban "pametan" koncept modernog domaćinstva i življenja u njemu. Sistemom se može upravljati na različite načine a sve u skladu sa životnim stilom, potrebama i navikama korisnika. Takođe, važna karakteristika ovakvog sistema je što pruža mogućnost potpunog prilagođavanja svakodnevnim različitim aktivnostima i raspoloženju svakog ukućana uzimajući pri tome u obzir optimum u pogledu odnosa korisnosti i ostvarenja veće energetske efikasnosti.



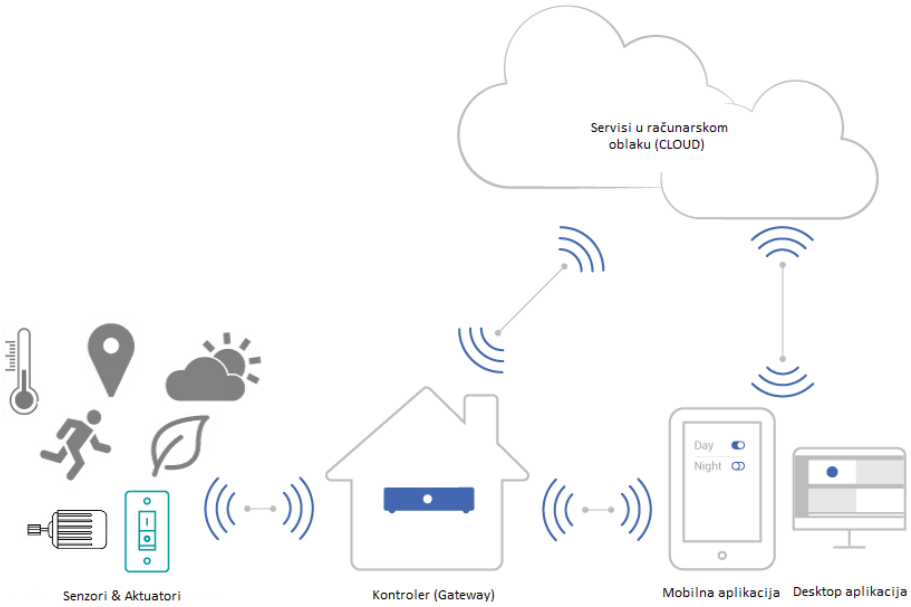
Sl.1. Aspekti IoT-a.

U nastavku navodimo primer sistema za kontrolu pametne kuće.

Student Fakulteta Tehničkih Nauka, odseka za Računarsku Tehniku i Računarske Komunikacije, nakon sjajnog, dugo isčekivanog dana provedenog na predavanjima i vežbama iz predmeta Osnovi Računarskih Mreža 2, u povratku sa fakulteta stiže kući. Neposredno pred ulazak vrata stana se sama otvaraju, pri ulasku ambijentalno osvetljenje koje reflektuje trenutno raspoloženje (sreću i zadovoljstvo) se uključuje, nakon ulaska vrata se sama zatvaraju i zaključavaju. Klima uređaj u kući se na zahtev putem aplikacije na mobilnom telefonu aktivirao neposredno pred odlazak sa fakulteta kako bi temperatura pri dolasku kući u prostorijama bila potpuno prijatna, pri ulasku u dnevnu sobu TV prijemnik se automatski startuje sa obaveštenjem da za 5 minuta počinje omiljena emisija, roletne na prozorima automatski se prilagođavaju dobu dana i količini dnevne svetlosti u prostoriji. Nakon odgledane emisije, na zahtev putem glasovne komande TV se gasi, i student sa nepresušnom željom za potvrđivanjem i priznanjem kreće da radi domaće zadatke za bonus poene i ponavlja pređeno gradivo kako bi spremno dočekao još jedan novi dan pun akademskih izazova.



Sl.2. Pametna kuća.



Sl.3. Komponente IoT sistema pametne kuće.

ZADATAK VEŽBE 1

Formirati timove

Identifikovati komponente sistema koji obavlja zadatu funkciju

2. MODEL UREĐAJA

2.1. Modeli uređaja u popularnim IoT tehnologijama

Kao što je u prethodnom poglavlju navedeno, osnovne komponente koje čine jedan IoT ekosistem, koje mogu biti predstavljene uz pomoć odgovarajućih modela uređaja su:

- Model - Krajnji uređaji. A to su:

  _____?
- Model - Kontroler sistema. Koji može biti:

  _____?
- Model - Korisničke aplikacije, koje omogućuju:

  _____?
- Model - Servisi u oblaku, koji omogućuju:

  _____?

IoT sistem funkcioniše kroz koncept međusobnog umrežavanja širokog spektra gore pomenutih (modela) uređaja. Povezivanjem takvih raznorodnih uređaja, otvaraju se mogućnosti za izgradnju kvalitativno novih sistema, sa novim primenama, kao što su pametni gradovi, pametna putna infrastruktura, pametne kuće i drugo. Osnovu IoT-a čine različite tehnologije bežičnog umrežavanja poput Zigbee, Z-Wave, Bluetooth, Bluetooth Low Energy (BLE), WiFi itd.

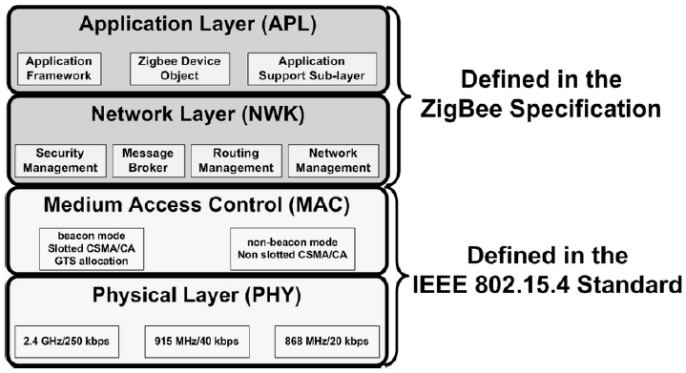
U nastavku je dat kratak pregled danas dominantnih tehnologija bežičnog umrežavanja sa osvrtom na njihovu primenu u IoT sistemima. Sticanjem osnovnih znanja koja su zajednička za različite tehnologije omogućen je jednostavan prelaz između istih.

2.1.1. Zigbee

ZigBee je bežična tehnologija razvijena kao otvoreni globalni standard. Standard treba da odgovori na jedinstvene zahteve u pogledu niske cene i male potrošnje energije bežičnih uređaja. ZigBee je zasnovan na IEEE 802.15.4 standardu personalnih računarskih mreža - PAN (*Personal Area Network*), koji koristi nelicencirane frekventne opsege: 2.4 GHz, 900 MHz i 868 MHz.

Koristi se u slučajevima u kojima nije neophodna velika bitska brzina prenosa podataka. Dakle, Zigbee se ne može koristiti za prenos signala osetljivih na vremensko

kašnjenje (video, zvuk). Prednost protokola iz ove grupe se ogleda u niskoj potrošnji, koja obezbeđuje dugačak vek baterije uređaja. Zigbee uređaji mogu da komuniciraju u različitim topologijama mreže. Fizički i MAC podsloj sloja veze podataka definisani su standardom IEEE 802.15.4, dok Zigbee alijansa definiše više slojeve protokola. U zavisnosti od namene protokola, razlikujemo različite implementacije viših slojeva – aplikativne profile, koje nudi Zigbee alijansa (smart home, smart energy, itd).



Sl.4.Slojevi Zigbee protokol steka.

2.1.2. Z-Wave

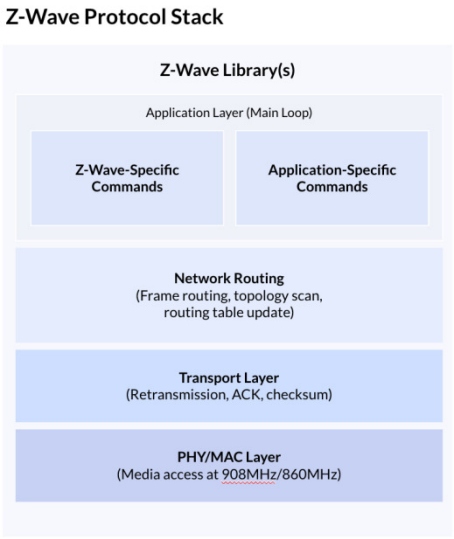
Z-Wave tehnologija je vlasnička tehnologija razvijena od danske firme Zensys, koju je kasnije kupila firma Sigma Designs. Oni su vlasnici tehnologije i proizvode Z-Wave čipove. Z-Wave alijansa je osnovana 2005. godine, čini je više od 450 kompanija, i zadužena je za sertifikaciju proizvoda. Fizički i MAC sloj koji koristi Z-Wave su definisani ITU-T G.9959 standardom, dok informacije o višim slojevima nisu javno dostupne.

Z-Wave je bežični komunikacioni protokol koji se pretežno koristi u sistemina kućne automatizacije. Ova tehnologija se koristi u kućnim elektronskim uređajima poput alarma, klima-uređaja, termostata, kao i u audio i video opremi za elektronski nadzor, itd. U okvirima Z-Wave MESH zasnovane mrežne topologije, svaki uređaj u mreži je sposoban za slanje i primanje upravljačkih naredbi kao i podataka.

Uređaji zasnovani na ovoj tehnologiji ne koriste iste frekvencijske opsege kao i drugi kućni uređaji koji obično rade na 2,4 GHz. Opsezi koje koristi Z-Wave tehnologija su 908,42 MHz u Sjedinjenim Državama i 868 MHz u Evropi. Dakle, Z-Wave uređaji neće ometati druge uređaje za domaćinstvo. Upravo zbog prethodno navedene činjenice, Z-Wave uređaji imaju veći doseg signala. Na doseg signala utiču brojni faktori, pre svega prisustvo zidova u blizini. Tipično, spektar dosega signala iznosi oko 30 metara u unutrašnjosti objekta i 100 metara na otvorenom. Proširenje dosega je omogućeno dodavanjem dodatnih Z-Wave uređaja u mrežu. Pošto svi Z-Wave uređaji predstavljaju repetitore, svaki put kada se signal ponavlja, dobija se još oko 30 dodatnih metara dosega. Do tri dodatna uređaja – repetitora skoka (*Hop*) se može koristiti za proširenje signala pre nego što protokol prekine signal (nazvan " *Hop Kill"*),

sto znači da maksimalan broj repetitora na putanji od izvora do odredišta poruke iznosi 4.

Pitanje: Na osnovu prethodno navedenog, da li je potrebna direktna vidljivost između izvora i odredišta koji koriste ovu tehnologiju?



Sl.5.Slojevi Z-Wave protokol steka.

2.1.3. BLE

Bluetooth tehnologija razvijena je u Eriksonu, tokom prve polovine devedesetih godina dvadesetog veka. Prva verzija specifikacije pretvorena je u IEEE 802.15.2 standard, a danas standard održava Bluetooth SIG (Special Interest Group), grupa kompanija koja sprovodi i sertifikaciju uređaja. Postoje dva tipa Bluetooth tehnologije – osnovni BR/EDR (Basic Rate / Enhanced Data Rate; većim delom pokrivena IEEE 802.15.1 standardom) i BLE (Bluetooth Low Energy), namenjen IoT primenama.

Bluetooth tehnologija specificirana standardom IEEE 802.15.1 namenjena je pre svega zameni kablova između računara, prenosu audio signala i sl. Za primene u IoT oblasti, činjenica da svi uređaji moraju periodično osluškivati signal od vodećeg uređaja u pikonetu predstavlja problem, jer se troši veća energija baterije. Zbog toga je Bluetooth SIG specifikacija definisala varijantu ove tehnologije poznatu kao BLE (*Bluetooth Low Energy*), koja nije pokrivena IEEE standardom. BLE tehnologija nije namenjena prenosu veće količine podatka, već joj je prvenstvena namena prenos kraćih paketa, bez održavanja stalne konekcije. Po cenu brzine prenosa, postignuta je mogućnost upotrebe manjih baterija (dugmastih umesto AAA, npr.).

Na slici 6. prikazane su glavne karakteristike Bluetooth i BLE tehnologija.

Technical Specification	Classic Bluetooth	Bluetooth Low Energy
Frequency	2400 to 2483.5 MHz	2400 to 2483.5 MHz
Modulation Technique	Frequency Hopping	Frequency Hopping
Modulation Scheme	GFSK	GFSK
Modulation Index	0.35	0.5
Number of Channels	79	40
Channel Bandwidth	1 MHz	2 MHz
Nominal Data Rate	1 - 3 Mbps	1 Mbps
Application Throughput	0.7 - 2.1 Mbps	< 0.3 Mbps
Nodes / Active Slaves	7	Unlimited
Security	56 - 128 bit	128-bit AES
Robustness	FHSS	FHSS
Voice	Capable	Not Capable

Sl.5.Bluetooth vs. BLE.

2.2. Oglašavanje funkcionalnosti uređaja

Nakon identifikacije svih komponenti IoT sistema koji obavlja datu funkciju iz poglavlja 1, potrebno je napomenuti da fizički uređaji tj. njihove reprezentacije u vidu programskih modela, ne poseduju unapred nikakvo znanje jedni o drugima, već samo poznaju format poruka koje mogu da razmenjuju. Stoga dati uređaji moraju da poseduju funkcionalnosti oglašavanja tj. otkrivanja ostalih komponenti sistema, dok je format poruka između ovih komunikacionih entiteta unapred poznat.

Glavna ideja realizacije je da se na dinamičan način krajnji uređaji u vidu senzora i aktuatora povezuju na centralni kontroler i da korisnik putem korisničke aplikacije može da im pristupa putem funkcionalnosti implementiranih u okvirima centralnog kontrolera za tu svrhu. Pristupanjem centralnom kontroleru korisnik da može da proveri njihovo stanje, tj. vrednosti svih parametara, informacije o uređaju i menja vrednosti parametara.

Više o protokolu koji omogućuje krajnjim tj. upravljanim uređajima da pri priključivanju u mrežu objave svoje prisustvo i funkcionalnosti kontroleru tj. kontrolnim tačkama, kao i takođe, istima da pretražuju mrežu i pronađu upravljane uređaje od interesa biće dato u poglavlju 4.

2.3. Izvršavanje komandi

Korisnik sistema putem korisničke aplikacije pristupa krajnjim uređajima, tako što šalje komande centralnom kontroleru. Korisnička aplikacija pruža CLI (*Command Line Interface*) korisniku za rukovanje.

Kroz CLI interfejs aplikacije korisnik može da:

- Pregleda opis svih registrovanih uređaja

- Pregleda stanje svih registrovanih uređaja

- Zadaje komande centralnom kontroleru

Komande mogu biti definisane kao namenski jezik, za koji je napravljen interpreter na korisničkoj strani. Nakon što se utvrdi ispravnost komandi, one se šalju centralnom kontroleru uz pomoć MQTT protokola (o istome više reči će biti dato u narednom poglavlju). Rezultate izvršenja komandi centralni kontroler šalje korisniku korišćenjem istog protokola.

Više reči o izvršavanju komandi će biti dato u poglavlju 6.

ZADATAK VEŽBE 2

Osmisliti modele uređaja koji su potrebni u sistemu

3. KOMUNIKACIONI MODEL

3.1. MQTT protokol

MQTT (MQ Telemetry Transport) protokol predstavlja protokol aplikativnog sloja ISO OSI protokol steka, koji služi za razmenu poruka između uređaja. Glavni aspekti koji ga čine i najzastupljenijim protokolom unutar IoT su:

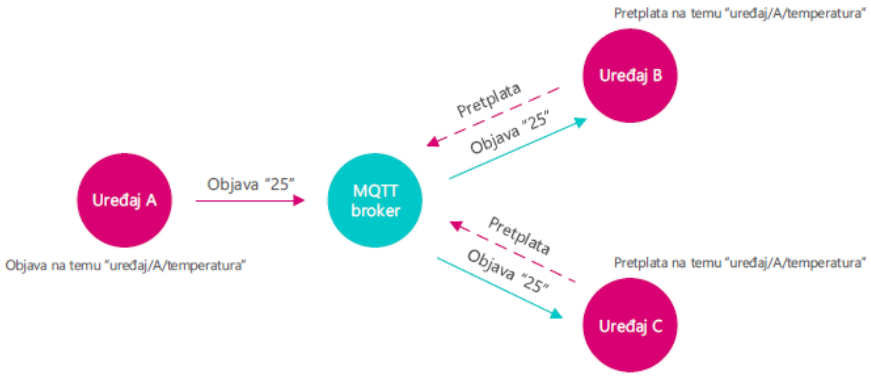
- asinhrona priroda,
- efikasna i brza razmena poruka,
- jednostavnost,
- kompatibilnost i fleksibilnost samog protokola.

Jednostavna implementacija komponenti ovog protokola, u sinergiji sa malom potrošnjom energije prilikom komunikacije, omogućava njegovo korišćenje na namenskim uređajima, što su i najbitnije osobine da ovaj protokol bude široko prihvaćen od strane IoT zajednice.

3.2. Obrasci za komunikaciju

3.2.1. Asinhrona komunikacija (mehanizam pretplate i objave)

MQTT protokol je zasnovan na principu pretplate i objave (*publish-subscribe*) posredstvom komponente zvane **broker**. Klijenti se pretplaćuju i objavljuju određeni informacioni sadržaj na takozvane **teme** (*topics*) za koje su zainteresovani. Po MQTT protokolu, broker je centralna komponenta mreže, i njegov zadatak je da, poruke koje pristižu, pravilno preusmeri onim klijentima koju su na tu temu pretplaćeni.



Sl.6.MQTT – pimer razmene poruka.

Slika 6 prikazuje primer razmene poruka posredstvom MQTT protokola gde u komunikaciji učestvuju 3 klijenta i broker. Uređaji B i C su pretplaćeni na temu na koju uređaj A objavljuje informacioni sadržaj. Samim tim, objavljeni sadržaj dolazi do uređaja B i C posredstvom brokera. Prednost ovog pristupa leži u činjenici da se omogućava jednostavno distribuiranje iste poruke do svih zainteresovanih korisnika. Pri tome, nije potrebno da uređaj koji objavljuje poruku zna detalje o pretplatnicima (poput adrese, stanja veze itd.).

3.2.2. Sinhrona komunikacija (mehanizam zahteva i odgovora)

Asinhrona komunikacija zasnovana na mehanizmu pretplate i objave poruka jeste osnovna osobina MQTT protokola i to zadovoljava zahtev za razmenom informacija. Međutim, slanje zahteva i dobijanje odgovora ne predstavlja asinhronu komunikaciju i po originalnoj specifikaciji protokola takav vid komunikacije "standardno" nije pokriven (što ne znači da realizacija ovakvog vida komunikacije nije moguća; moguće je realizovati ovakav vid komunikacije kroz sam informacioni sadržaj unutar MQTT poruka). Najnovija verzija MQTT protokola V5 uvodi svojstvo "response topic" koji omogućava ovakav vid komunikacije.

3.2.3. Teme u MQTT protokolu

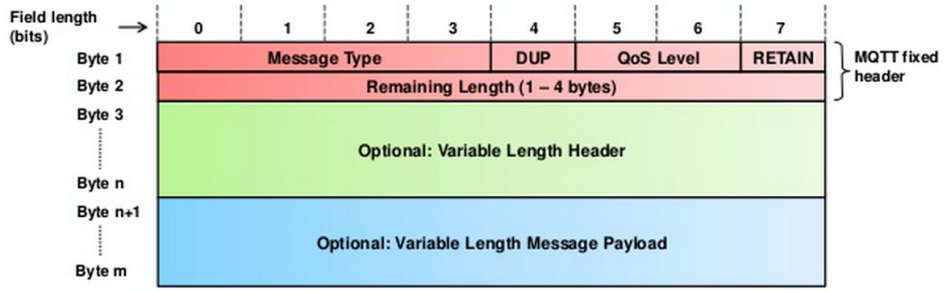
MQTT teme (UTF-8 enkodovani string) se formiraju na osnovu hijerarhijski organizovanih komponenti koje su razdvojene simbolom "/" (Slika 7). Zbog ove osobine, moguće je definisati teme tako da uključuju različite informacije koje najbolje odgovaraju određenoj svrsi, a u cilju lakšeg pronalaženja određene funkcionalnosti i bolje kontrole pristupa. Maksimalna veličina teme je 32,767 karaktera. Prilikom pretplaćivanja, moguće je koristiti i simbol "+" i simbol "#". Svaka komponenta u strukturi teme može biti zamenjena simbolom "+", na taj način je moguće pretplaćivanje na grupu tema u zavisnosti od komponente koju ovaj simbol menja. Simbolom "#" se može menjati jedna ili više krajnjih komponenti teme. Ovaj simbol se mora naći na kraju teme.



Sl.7.MQTT tema.

3.2.4. Struktura poruka

Format MQTT poruke dat je na slici 8. Sve poruke sadrže informaciju o tipu poruke, QoS nivou koji poruka zahteva (QoS se u MQTT protokolu odnosi na garanciju isporuke poruke), kao i informaciju da li se zahteva od brokera da poruku sačuva i isporuči naknadno povezanim klijentima (retain polje). Opciono, poruka može imati i zaglavlje i podatke.



Sl.8.Format MQTT poruke.

3.2.5. Kvalitet nivoa usluge(QoS)

MQTT dostavlja poruke prema takozvanom tipu kvaliteta usluge. Tri tipa kvaliteta usluge su definisana:

1. *QoS 0*: Isporuka poruke može izostati, ali se može dogoditi da ista poruka bude dostavljena više puta.

2. *QoS 1*: Ovaj tip kvaliteta usluge garantuje da će poruka biti isporučena barem jednom. Mada, i u ovom slučaju se može desiti da poruka bude dostavljena više puta.

3. *QoS 2*: Kod ovog tipa kvaliteta usluge, nije prihvatljiv ni gubitak poruke, ni da poruka bude dostavljena više puta.

ZADATAK VEŽBE 3

Osmisliti komunikacioni model

KONTROLNA TAČKA 1

Dizajn specifikacija sistema

4. OTKRIVANJE MREŽE I PRIKLJUČIVANJE UREĐAJA

4.1. Načini oglašavanja u popularnim IoT tehnologijama

Kao što je u prethodnom poglavlju navedeno, MQTT protokol se koristi za razmenu informacija među komponentama sistema nakon sto se iste prvo uspešno povežu. Pre uspostavljanja veze između komponenti sistema, potrebno je razmotriti načine oglašavanja prisustva i funkcionalnosti krajnjih uređaja kontroleru tj. kontrolnim tačkama, kao i obrnuto.

4.2. Oglašavanje WiFi mreže

Jedna od najčešće korišćenih metoda kako bi se prenele Wi-Fi informacije do krajnjeg uređaja čiji operativni system (*OS*) poseduje TCP/IP podršku je dovođenje uređaja u režim rada pristupne tačke (*AP*, *Access Point*). Zatim se korisnik upotrebom Internet pretraživača tj. aplikacije povezuje sa uređajem na kome je prethodno pokrenut poslužilac, i kroz ponuđenji grafički interfejs prosleđuje i dobija informacije putem pristupne tačke. Priključivanje krajnjeg uređaja internoj mreži centralnog kontrolera predstavlja proces, tokom kog, obe strane predstavljaju svoje informacije koje su potrebne kako bi se ovaj proces završio uspešno. Nakon ovog procesa, potpuna komunikacija između centralnog kontrolera i uređaja je omogućena. Sam proces se razlikuje u zavisnosti od režima rada. U režimu rada, kada su centralni kontroler i krajnji uređaj u istoj mreži, za predstavljanje informacija koristi se SSDP protokol. U režimu rada, kada je centralni kontroler deo Internet oblaka, obično se koristi HTTP protokol za predstavljanje informacija.

Na linku navedenom ispod, objašnjen je postupak dovođenja platforme Raspberry Pi u režim rada pristupne tačke.

https://www.raspberrypi.org/documentation/configuration/wireless/access-point.md

4.3. SSDP protokol

SSDP protokol (*Simple Service Discovery Protocol*) predstavlja deo UPnP (*Universal Plug and Play*) protokola, kao protokol za otkrivanje uređaja na mreži. UPnP protokolom su definisane dve grupe uređaja na mreži: uređaji kojima se upravlja i kontrolne tačke. SSDP protokol omogućuje upravljanom uređaju da pri priključivanju mreži, objavi svoje prisustvo kontrolnim tačkama. Takođe, omogućuje kontrolnim tačkama da pretražuju mrežu i pronađu upravljane uređaje od interesa.

Upravljani uređaj koji se priključuje mreži objavljuje svoje prisustvo višeznačnim upućivanjem poruke (eng. *multicast*), dok kontrolna tačka osluškuje iste. Prilikom

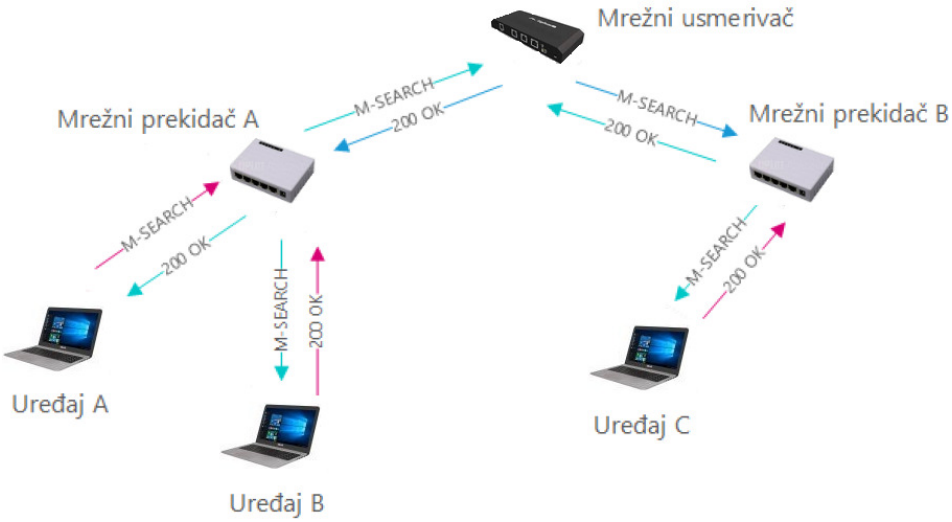
isključenja sa mreže, upravljani uređaj šalje poruke pomoću kojih poništava prethodno poslate objave. Pri priključivanju mreži, kontrolna tačka može da pošalje višeznačnim upućivanjem poruku kojom pretražuje upravljane uređaje na mreži. Svi upravljani uređaji moraju da osluškuju grupu višeznačnog upućivanja i da odgovore na ovakvu poruku formatom definisanim protokolom.

Poruke se razmenjuju preko UDP protokola. Neke od poruka su navedene ispod:

1. NOTIFY (ssdp:alive) – Šalje se pri priključivanju upravljanog uređaja mreži. Sadrži tip objave, identifikator objave, putanju do detaljnih informacija i trajanje važnosti objave.

2. NOTIFY (ssdp:byebye) – Šalje se pri isključivanju upravljanog uređaja sa mreže.

3. M-SEARCH – Šalje se od strane kontrolne tačke kako bi pretražila mrežu. Odgovor na ovu poruku se šalje ukoliko upravljani uređaj ispunjava uslov M-SEARCH poruke. Pored putanje do detaljnih informacija, sadrži i kratak opis upravljanog uređaja.



Sl.9.SSDP pretraživanje mreže.

Slika 9 prikazuje slanje M-SEARCH poruke sa višeznačnim (*multicast*) upućivanjem uređaja A. Uređaji B i C pripadaju grupi uređaja koju će dobiti M-SEARCH poruku i na nju odgovaraju, nakon čega, uređaj A zna da se na mreži nalaze uređaji B i C.

4.3.1. NOTIFY poruka

NOTIFY poruka se šalje pri priključivanju upravljanog uređaja na mrežu ili prilikom isključivanja upravljanog uređaja sa mreže. Inicijalno objavljivanje ove poruke opisuje uređaj, opciono, integrisane (eng. *embedded*) uređaje koje sadrži kao i usluge koje obezbeđuje kontrolnim tačkama na mreži. Zbog nepouzdane prirode UDP protokola, poruka se šalje više puta sa pauzama od nekoliko stotina milisekundi. Slika

10 prikazuje poruku NOTIFY tipa **ssdp:alive** koja upravljani uređaj šalje prilikom priključivanja mreži. Unutar polja **LOCATION**, nalazi se adresa do detaljnih informacija u XML formatu koje kontrolna tačka može dobaviti slanjem GET zahteva iz HTTP protokola.

Primer NOTIFY (ssdp:alive) poruke sa poljima zaglavlja je dat ispod:

NOTIFY * HTTP/1.1

HOST: 239.255.255.250:1900

CACHE-CONTROL: max-age = Trajanje važnosti objave.

LOCATION: Adresa do detaljnih informacija.

NT: Tip objave.

NTS: ssdp:alive

SERVER: Operativni sistem, verzija protokola.

USN: Identifikator objave.

BOOTID.UPNP.ORG: Broj priključivanja uređaja mreži.

CONFIGID.UPNP.ORG: Konfiguracioni broj uređaja.

SEARCHPORT.UPNP.ORG: Mrežni prolaz na kome uređaj osluškuje

MSEARCH poruke poslate jednoznačnim upućivanjem.

Primer **NOTIFY ssdp:byebye** poruke koja se šalje prilikom isključivanja upravljanog uređaja sa mreže sa poljima zaglavlja je dat ispod:

NOTIFY * HTTP/1.1

HOST: 239.255.255.250:1900

NT: Tip objave.

NTS: ssdp:byebye

USN: Identifikator objave.

BOOTID.UPNP.ORG: Broj priključivanja uređaja mreži.

CONFIGID.UPNP.ORG: Konfiguracioni broj uređaja.

4.3.2. M-SEARCH poruka

Kada su krajnji uređaj i centralni kontroler u istoj mreži, mrežu pretražuje uređaj slanjem **M-SEARCH** poruke definisane SSDP protokolom. Poruka se može slati sa višeznačnim upućivanjem svim uređajima koji osluškuju ove poruke i sa jednoznačnim upućivanjem jednom specifičnom uređaju čija identifikacija se nalazi unutar **ST** zaglavlja.

Ispod je dat primer **M-SEARCH** poruke sa poljima zaglavlja na koju odgovaraju oni uređaji koji ispunjavaju uslov definisan upravo u **ST** zaglavlju.

M-SEARCH * HTTP/1.1

HOST: 239.255.255.250:1900

MAN: ssdp:discover

MX: Maksimalno vreme čekanja, u sekundama, pre slanja odgovora. Ova vrednost mora biti između 1 i 5. Odgovor se šalje posle broja sekundi koji je nasumično izabran u opsegu od 0 do broja koji je definisan ovim poljem.

ST: Uslov pretraživanja.

USER-AGENT: Operativni sistem, verzija protokola.

CPFN.UPNP.ORG: Prijateljsko ime kontrolne tačke.

CPUUID.UPNP.ORG: UUID kontrolne tačke.

4.3.3. RESPONSE poruka kao odgovor na M-SEARCH poruku

Kao i kod **NOTIFY** poruke, druga strana u komunikaciji šalje odgovor na **M-SEARCH** poruku sa zaglavljem **LOCATION** gde se nalazi putanja do njegovih detaljnih informacija u XML formatu koje kontrolna tačka može dobaviti slanjem GET zahteva iz HTTP protokola.

HTTP/1.1 200 OK

CACHE-CONTROL: max-age = Trajanje važnosti objave.

DATE: Vreme kada je odgovor generisan.

EXT: Ovo polje ostaje prazno. Postoji zbog podržavanja prethodnih verzija UPnP protokola.

LOCATION: Adresa do detaljnih informacija.

SERVER: Operativni sistem, verzija protokola.

ST: Uslov pretraživanja.

USN: Identifikator objave.

BOOTID.UPNP.ORG: Broj priključivanja uređaja mreži.

CONFIGID.UPNP.ORG: Konfiguracioni broj uređaja.

SEARCHPORT.UPNP.ORG: Mrežni prolaz na kome uređaj osluškuje MSEARCH poruke poslate jednoznačnim upućivanjem.

4.4. Opis funkcionalnosti uređaja

Osobine krajnjeg uređaja opisuju dve JSON strukture, a to su opis uređaja i stanje uređaja. Opis se sastoji od: grupe uređaja, identifikacije uređaja, servisa i karakteristika svakog parametra koji definiše krajnji uređaj. Stanje se sadrži od parova parametar-vrednost.

Primer opisa uređaja:

```
{
    "id" : "light_bulb_1",
    "group" : "bulb",
    "ContactService" :
    {
            "State" : "T/F"
    },
    "LightService" :
    {
            "DimLevel" : "0~100",
            "State": "ON/OFF",
            "Color": "RGB/HS"
    }
}
```

Za parametre imamo dve vrste liste vrednosti a to su opseg i niz. Opseg: startValue ~ endValue. Niz: value1/value2/…/valueN.

Parametri koji mogu samo da se pročitaju, ali ne i da se modifikuju od strane korisnika su obeleženi tako što se pored njihove liste vrednosti dodaje "|r" string.

Opis jednog multifunkcionalnog senzora:

```
{
    "id" : "weather_sensor_1",
    "group" : "sensors",
    "TempService" :
    {
            "Temperature" : "-50~50|r",
    },
    "HumidityService":
```

```
        {
                "Humidity" : "900~1100lr"
        }
}
```

Primer stanja uređaja:

```
{
    "id" : "light_bulb_1",
    "group" : "bulb",
    "ContactService" :
    {
                "State" : "T"
    } "
    LightService" :
    {
                DimLevel" : "55",
                "State": "ON",
                "Color": "RGB"
    }
}
```

ZADATAK VEŽBE 4A

Realizacija modula za oglašavanje kontrolera i kontrolu pristupa

ZADATAK VEŽBE 4B

Realizacija modula za pristupanje mreži na strani uređaja

5. ODRŽAVANJE MREŽE

5.1. Praćenje stanja mreže i uređaja na strani kontrolera

5.1.1. Čuvanje informacija o uređajima u mreži

Centralni kontroler predstavlja posrednika u komunikaciji između korisnika i krajnjih uređaja. Skladišti informacije o opisu i stanju svakog registrovanog uređaja. Korisnik ne komunicira direktno sa uređajima već sve informacije o uređajima dobija od centralnog kontrolera, koji to čuva u svojoj internoj bazi. Na istom fizičkom uređaju na kom se pokreće softver za centralni kontroler, pokreće se i MQTT broker.

5.1.2. Otkrivanje nedostupnog uređaja

Tokom interakcije sa krajnjim uređajem, centralni kontroler može primetiti da je došlo do prekida veze (usled kvara tj. otkaza uređaja) bez da je krajnji uređaj poslao poruku stanja sa atributom koji označava da nije na mreži.

Ako dođe do ovog slučaja, zadatak centralnog kontrolera je da proglasi uređaj za neaktivnim, uklani ga iz liste registrovanih uređaja i da šalje NOTIFY poruku definisanu SSDP protokolom. NOTIFY poruka se šalje kako bi krajnji uređaj, u slučaju da se pojavi na mreži, po prijemu ove poruke, mogao da uspostavi vezu sa centralnim kontrolerom.

Slično, krajnji uređaj zna kada se centralni kontroler odjavio, kako bi mogao da ponovo započne proces pretrage. Pređašnja funkcionalnost je obezbeđena u okvirima MQTT protokola. Krajnji uređaj će ponovo pokušati da uspostavi vezu sa centralnim kontrolerom koristeći prethodno utvrđene kredencijale koje je dobio tokom procesa priključivanja internoj mreži centralnog kontrolera. Ukoliko je uspostava veze neuspešna, krajnji uređaj počinje osluškivanje mreže čekajući NOTIFY poruku defnisanu SSDP protokolom, u slučaju kada su krajnji uređaj i centralni kontroler u istoj mreži. U slučaju kada je centralni kontroler deo Internet oblaka, krajnji uređaj dobavlja opis centralnog kontrolera sa unapred definisane putanje slanjem HTTP GET zahteva.

5.2. Načini rada uređaja

5.2.1. Spavajući uređaji

Uzimajući u obzir da se krajnji uređaji mogu napajati baterijom, i da mogu samo da objavljuju svoja stanja bez mogućnosti da se ta stanja menjaju sa strane centralnog kontrolera, uređaji ne moraju imati konstantnu vezu za centralnim kontrolerom i takvi uređaji se nazivaju spavajućim uređajima. Ovakav tip uređaja definiše vreme tokom kojeg je povezan sa centralnim kontrolerom i vreme koje provede u stanju kada nije povezan sa centralnim kontrolerom.

5.2.2. Uvek aktivni uređaji

Uređaji čija stanja kontroler u datom vremenskom trenutku može menjati nazivaju se aktivnim uređajima.

Dakle, u svakom trenutku centralni kontroler poseduje informaciju o trenutnom stanju krajnjeg uređaja priključenog u mreži: spavajuće i aktivno.

ZADATAK VEŽBE 5A

Realizacija komponente za kontrolu stanja veze na strani uređaja

ZADATAK VEŽBE 5B

Realizacija komponente za upravljanje lokalnom mrežom na strani kontrolera

ZADATAK VEŽBE 5C

Čuvanje informacija o priključenim uređajima na strani kontrolera

KONTROLNA TAČKA 2

Demonstracija različitih tipova uređaja

6. UPRAVLJANJE UREĐAJIMA

6.1. Funkcionalnost uređaja

6.1.1. Izvršavanje komandi

Krajnji uređaji nude mogućnost menjanja osobina funkcionalnosti kao i dobavljanja informacija o uređaju kroz aplikativnu programsku spregu. Zahtevi se mogu slati npr. na teme sa oznakom funkcionalnosti i bez oznake funkcionalnosti. Zahtevi koji se šalju na temu bez oznake funkcionalnosti su namenjeni samom krajnjem uređaju, sa ciljem dobavljanja informacija o uređaju i njegovim funkcionalnostima.

Primeri zahteva koji se šalju na temu bez oznake funkcionalnosti su dati ispod:

- **GetDevInfo** zahtev – Ovim zahtevom se dobavljaju detaljne informacije o uređaju.
- **SetDevInfo** zahtev – Namena ovog zahteva je promena neke informacije o uređaju nad kojom je ova promena dozvoljena.
- **GetServiceList** zahtev – Zahtev služi za dobavljanje liste svih funkcionalnosti sa njihovim osobinama iz onog skupa funkcionalnosti koji je definisan ovim zahtevom.

Zahtevi koji se šalju na temu sa oznakom funkcionalnosti namenjeni su promeni osobina kao i dobavljanju nekih informacija o toj funkcionalnosti. Primeri takvih zahtevi su navedeni ispod:

- **GetPropertyList** zahtev – Zahtev je namenjen dobavljanju detaljnih informacija o osobinama funkcionalnosti.
- **GetPropertyValue** zahtev – Ovaj zahtev se koristi za dobijanje trenutne vrednosti one osobine funkcionalnosti koja je definisana ovim zahtevom.
- **SetPropertyValue** zahtev – Ovim zahtevom se menja vrednost osobine funkcionalnosti koja je definisana kroz parametre ovog zahteva.

6.1.2. Odgovor na upit stanja

Pri promeni osobina bilo koje funkcionalnosti tj. upita stanja, krajnji uređaj šalje poruku obaveštenja, kako bi centralni kontroler bio obavešten o toj promeni/stanju. Naziv ovakve poruke obaveštenja može biti npr. **PropertyChanged**. Pored ove poruke obaveštenje, krajnji uređaj šalje i poruku obaveštenja pri svakom uspešnom pokretanju npr. pod nazivom **DevReboot**.

6.2. Korisnička aplikacija

Kao što je već napomenuto u poglavlju 2, korisniku sistema putem korisničke aplikacije je omogućeno da pristupa krajnjim uređajima, tako što šalje komande centralnom kontroleru. Osnovni zadatak korisničke aplikacije je da pruži korisniku uvid u trenutno stanje krajnjih uređaja i da omogući udaljenu kontrolu (*remote control*).

Kroz CLI interfejs aplikacije moguć je:

- Pregled opisa svih registrovanih uređaja

- Pregled stanja svih registrovanih uređaja

- Zadavanje komandi centralnom kontroleru

Komande mogu biti definisane kao namenski jezik, za koji je napravljen interpreter na korisničkoj strani. Nakon što se utvrdi ispravnost komandi, one se šalju centralnom kontroleru uz pomoć MQTT protokola. Rezultate izvršenja komandi centralni kontroler šalje korisniku korišćenjem istog protokola.

6.2.1. Prikaz trenutnog stanja uređaja

U nastavku sledi primer komandi za prikaz trenutnog stanja uređaja u formatu: *sintaksa komande - svrha komande*.

- ***SET devId.service.param value*** – postavlja parametar uređaja na zadatu vrednost.
- ***GET devId.info*** – zahtev za opis datog uređaja.
- ***GET devId.state*** – zahtev za stanje datog uređaja.
- ***GET *.info*** – zahtev za opis svih registrovanih uređaja.
- ***GET *.state*** – zahtev za stanje svih registrovanih uređaja.

Stanja i opis krajnjih uređaja koje korisnik dobavi od centralnog kontrolera se skladište na strani korisnika u njegovoj bazi podataka.

Poruke u komunikaciji su predstavljene korišćenjem JSON (Java Script Object Notation) struktura. Nakon što se komanda uspešno obradi ona se šalje centralnom kontroleru kao JSON struktura. Primer formata komandi je dat ispod:

SET komanda ->

```
{
    “command_type” : “SET”,
    “group”          : “devGroup”,
    “device”         : “devId”,
    “service”        : “serviceName”,
    “parameter”      : “paramName”,
```

```
    "value"              : "paramValue"
}
GET komanda →
{
    "command_type" : "GET",
    "json"               : "state | info",
    "device"             : "* | devId "
}
```

6.2.2. Izvršavanje komande

Zahtevi koji se šalju na temu sa oznakom funkcionalnosti mogu biti namenjeni takođe i izvršavanju akcija na krajnjim uređajima. Primeri takvih zahteva su navedeni ispod:

- **GetCommandList** zahtev – Ovaj zahtev služi za dobavljanje liste svih akcija koje je moguće izvršiti nad skupom funkcionalnosti uređaja.
- **ExecuteCommand** zahtev – Ovim zahtevom se izvršava tražena akcija sa ulaznim parametrima koji su prosleđeni kroz ovaj zahtev.

6.3. Uloga kontrolera.

Zadatak centralnog kontrolera je da komande primljene od strane korisnika prosledi krajnjim uređajima, kao i da informacije od značaja koje pristižu od krajnjih uređaja prosledi korisniku.

6.3.1. Keširanje informacije o trenutnom stanju uređaja

Kako se upravlja sistemom pametne kuće sa slika 1. i 2. iz poglavlja 1?

Centralni kontroler je zadužen za kontrolu čitavog sistema. Takođe, njegova uloga je i automatizacija. Centralni kontroler predstavlja posrednika u komunikaciji između korisnika i krajnjih uređaja. Korisnik ne komunicira direktno sa uređajima već sve informacije o uređajima dobija od centralnog kontrolera, koji to čuva u svojoj internoj bazi. Na istom fizičkom uređaju na kom se pokreće softver za centralni kontroler, pokreće se i MQTT broker.

Pruža takođe mogućnost skladištenja (keširanja) informacije o opisu i stanju svakog registrovanog uređaja.

ZADATAK VEŽBE 6A

Realizacija komunikacionog MQTT modula na strani uređaja

ZADATAK VEŽBE 6B

Realizacija komunikacionog MQTT modula na strani kontrolera

ZADATAK VEŽBE 6C

Realizacija konzolne korisničke aplikacije za prikaz stanja uređaja i kontrolu

KONTROLNA TAČKA 3

Demonstracija kontrole uređaja

7. NAPREDNA LOGIKA KONTROLERA

7.1. Grupisanje uređaja

Funkcionalnosti krajnjih uređaja je moguće povezati u grupe, čime odvajamo različite skupove povezanih funkcionalnosti.

7.2. Automatska podešavanja

7.2.1. Periodično izvršavanje komandi

Uređaji u IoT sistemu mogu da poseduju funkcionalnost periodične objave poruka tj. oglašavanja ili otkrivanja ostalih komponenti sistema itd.

7.2.2. Uslovno izvršavanje komandi

U okvirima IoT sistema moguće je definisati događaje (*event*) koji mogu predstavljati uslove za izvršavanje određenih operacija. U tom slučaju je moguće definisati npr. postupke tipa "sta treba da se desi kada nastupi određeno stanje tj. određena pobuda pojavi i kada ono/a završi". Na ovaj način, pomerajući granice onoga što IoT može da uradi, moguće je povezati IoT uređaje sa ciljem stvaranja moćnih lanaca komandi i operacija.

ZADATAK 7A

Realizacija modula za grupisanje uređaja i prikaz na korisničkoj aplikaciji

ZADATAK VEŽBE 7B

Realizacija modula za periodično izvršavanje komandi

ZADATAK VEŽBE 7C

Realizacija modula za uslovno izvršavanje komandi

KONTROLNA TAČKA 4

Demonstracija napredne logike kontrolera

LITERATURA

[1]